សប្តាហ៍ទី 2 · វិធាន និងការសម្រេចចិត្ត

Programming for Information Systems

នរណាជាអ្នកជ្រើសរើសវិធាន ហើយ program អនុវត្តវាដោយរបៀបណា?

ចាប់ផ្តើមថ្នាក់ · ចែករំលែកកិច្ចការស្រាវជ្រាវទី 1

ជាក្រុមបីនាក់ ចែករំលែកប្រព័ន្ធព័ត៌មានដែលអ្នកបានស្រាវជ្រាវ។

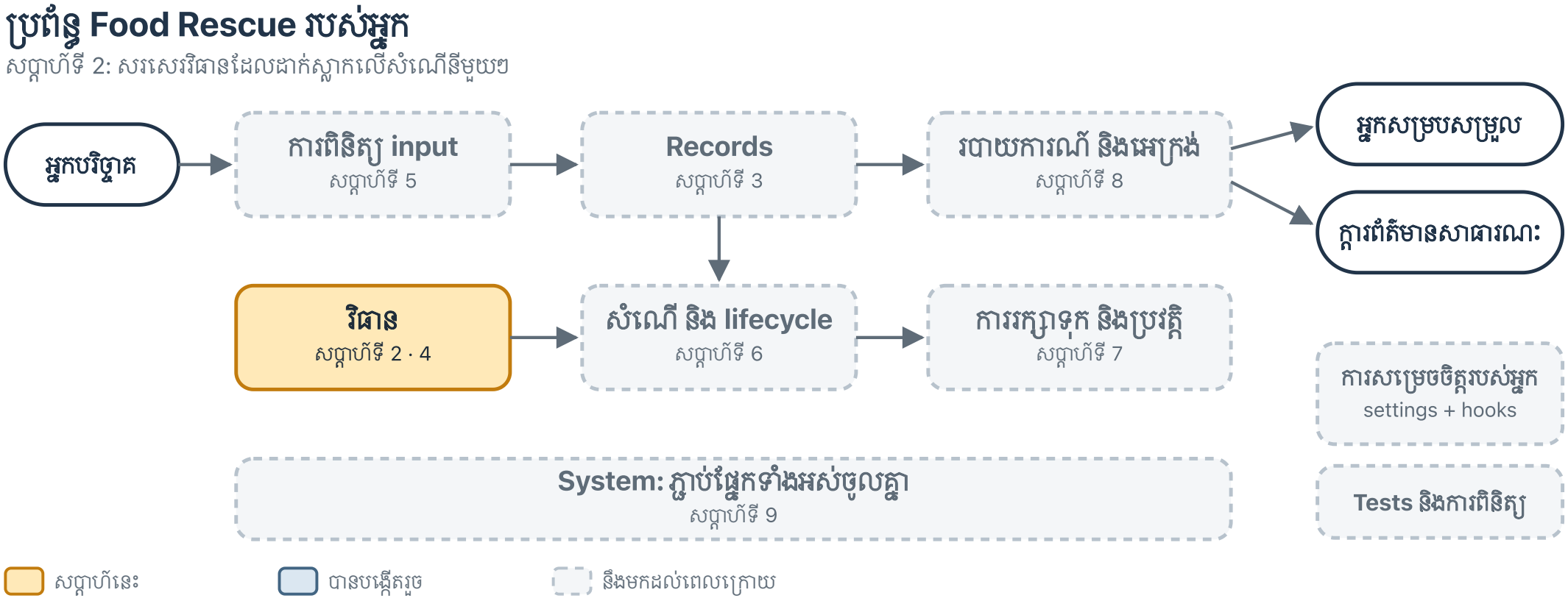
1. នរណាប្រើវា ហើយវារក្សាទុកទិន្នន័យអ្វីខ្លះ?
2. តើអ្នកបានរកឃើញ **វិធាន** អ្វី? វាត្រូវបានសរសេរទុក ឬមានតែនៅក្នុងក្បាលមនុស្ស?
3. តើអាចមានអ្វីខុស ហើយនរណានឹងរងគ្រោះ?

> 📝 កត់ត្រាគំនិតមួយពីមិត្តរួមថ្នាក់ ដែលអ្នកចង់ចងចាំ។

សប្តាហ៍នេះ

សំណួរ	តើប្រព័ន្ធធ្វើការសម្រេចចិត្តដោយរបៀបណា ហើយនរណាជាអ្នកជ្រើសរើសវិធាន?
Python	ការប្រៀបធៀប, `True` / `False`, `and` / `or` / `not`, `if` / `elif` / `else`, function តូចមួយដំបូង
គោលការណ៍ទី 2	វិធានច្បាស់លាស់: ដាក់ឈ្មោះឱ្យលេខ គ្របដណ្តប់គ្រប់ករណី test ព្រំដែន
អ្នកបង្កើត	`pickup_label` និង `urgency`

ប្រព័ន្ធរបស់អ្នក ត្រឹមសប្តាហ៍ទី 10



គំនិតសំខាន់: វិធានអាជីវកម្ម (business rule)

វិធានអាជីវកម្ម គឺជាការសម្រេចចិត្តអំពីរបៀបធ្វើការងារ។

- *"ពិន្ទុប្រឡងពី 50 ឡើងទៅ គឺជាប់។"*
- *"កុមារអាយុក្រោម 6 ឆ្នាំ ធ្វើដំណើរដោយឥតគិតថ្លៃ។"*

> **អ្នកសម្របសម្រួល:** "មិនទាន់អនុម័ត → **hold**។ បានអនុម័ត ហើយមាន **20 ចំណែក ឬច្រើនជាងនេះ** → **team**។ បើមិនដូច្នោះទេ → **standard**។"

- មនុស្សជាអ្នកជ្រើសរើសវិធាន។ Program អនុវត្តវាតាមរបៀបដដែល រាល់ដង។
- វិធានខុស ត្រូវបានអនុវត្តខុសរាល់ដង ដូច្នេះវិធានត្រូវតែ **សរសេរឱ្យច្បាស់លាស់** និង **test** នៅព្រំដែន។

មូលដ្ឋាន Python

របៀបដែល program សម្រេចចិត្ត

ការប្រៀបធៀបផ្តល់ `True` ឬ `False`

```
portions = 20
print(portions >= 20)        # True
print(portions > 20)         # False
print(portions == 20)        # True
print("bread" == "Bread")    # False
```

- `==` `!=` `<` `<=` `>` `>=`
- "20 ឬច្រើនជាងនេះ" គឺ `>= 20` ។ "ច្រើនជាង 20" គឺ `> 20` ។
- ការប្រៀបធៀប text ត្រូវតែដូចបេះបិទ: អក្សរធំ និងអក្សរតូច គឺខុសគ្នា។

ការផ្សំលក្ខខណ្ឌ: `and`, `or`, `not`

```
approved = True
portions = 25
print(approved and portions >= 20)        # ទាំងពីរត្រូវតែ True
print(not approved)                       # ផ្ទុយ
print(portions >= 20 or portions < 0)     # យ៉ាងហោចណាស់មួយ True
```

- `and`: គ្រប់ផ្នែកត្រូវតែ `True`។
- `or`: យ៉ាងហោចណាស់ផ្នែកមួយត្រូវតែ `True`។
- `not`: ប្តូរ `True` ទៅជា `False` ហើយផ្ទុយមកវិញ។

`if` និង `else`: ជ្រើសរើសអ្វីដែលត្រូវ run

```
temperature = 31
if temperature >= 30:
    print("Hot: keep the food in the shade")
else:
    print("Normal temperature")
print("This line always runs")
```

- បន្ទាត់ `if` បញ្ចប់ដោយសញ្ញា colon `:`។
- បន្ទាត់ដែលជារបស់វា ត្រូវ **ចូលបន្ទាត់ (indent)** 4 ដកឃ្លា។
- ការចូលបន្ទាត់មិនមែនសម្រាប់លម្អទេ: វាកំណត់ថាបន្ទាត់ណាជារបស់អ្វី។

`elif`: ពិនិត្យតាមលំដាប់ ទីមួយដែលត្រូវ គឺឈប់

```
mark = 72
if mark >= 80:
    grade = "A"
elif mark >= 65:
    grade = "B"
elif mark >= 50:
    grade = "C"
else:
    grade = "fail"
print(grade)
```

🤔 ពិន្ទុ 65 គត់ ទទួលបាននិទ្ទេសអ្វី? ចុះ 64 វិញ?

ដាក់ឈ្មោះឱ្យលេខរបស់អ្នក

```
TEAM_PICKUP_MIN_PORTIONS = 20      # អ្នកសម្របសម្រួលជាអ្នកសម្រេច

if portions >= TEAM_PICKUP_MIN_PORTIONS:
    ...
```

- លេខនៅក្នុងវិធាន គឺជា **ការសម្រេចចិត្ត**។
- ឈ្មោះមួយជួយឱ្យមនុស្ស **រកឃើញ** វា **យល់** វា និង **ប្តូរ** វានៅកន្លែងតែមួយ។
- ឈ្មោះជាអក្សរធំ គឺជា **constant**: program មិនប្តូរវាទេ ពេលវាកំពុង run។

Function ដំបូង: វេចខ្ជប់វិធានមួយ

```python
def is_hot(temperature):
    if temperature >= 30:
        return True
    return False

print(is_hot(31))    # True
print(is_hot(25))    # False
```

- `def` ចាប់ផ្តើម function; `temperature` គឺជា input របស់វា។
- `return` បញ្ជូនចម្លើយត្រឡប់មកវិញ ហើយ **បញ្ឈប់** function។
- ប្រព័ន្ធហៅវិធានដដែលនេះ សម្រាប់សំណើគ្រប់មួយ។

Test ព្រំដែន

```python
def needs_team(portions):
    if portions > 20:       # វិធាន: "20 ឬច្រើនជាងនេះ"
        return True
    return False

print(19, needs_team(19))   # False  ✓
print(20, needs_team(20))   # False  ✗  គួរតែជា True
print(21, needs_team(21))   # True   ✓
```

> មានតែករណី **ព្រំដែន** ទេ ដែលបង្ហាញ bug នេះ។ ត្រូវ test ជានិច្ច: **មួយខាងក្រោម ចំព្រំដែន និងមួយខាងលើ**។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
def label(p, a):
    if p > 20 and a == True:
        return "team"
    elif a == True:
        return "standard"
```

Version B

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    if not approved:
        return "hold"
    if portions >= TEAM_PICKUP_MIN_PORTIONS:
        return "team"
    return "standard"
```

គោលការណ៍ទី 2 · វិធានច្បាស់លាស់

- **ដាក់ឈ្មោះឱ្យលេខរបស់អ្នក**: constant មួយបង្ហាញការសម្រេចចិត្ត។
- **គ្របដណ្តប់គ្រប់ករណី**: គ្រប់ផ្លូវបញ្ចប់ដោយ `return`។
- **Test ព្រំដែន**: មួយខាងក្រោម ចំព្រំដែន និងមួយខាងលើ។

> ពេលអ្នកពិនិត្យ code ពី AI សួរថា: *តើមានអ្វីកើតឡើង ត្រង់ចំព្រំដែន ហើយពេលលក្ខខណ្ឌជា False?*

អ្នកបង្កើត: វិធានទៅយកទាំងពីរ

```
TEAM_PICKUP_MIN_PORTIONS = 20

def pickup_label(portions, approved):
    # មិនទាន់អនុម័ត → "hold"; ដល់ចំនួនអប្បបរមា → "team"
    # បើមិនដូច្នោះទេ → "standard"
    return "?"

fr.check("pickup_label", pickup_label,
         {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS})
```

- `urgency(hours_left)`: ≤ `URGENT_HOURS` → "urgent", ≤ `SOON_HOURS` → "soon", ក្រៅពីនេះ "later"។
- ការពិនិត្យ test ព្រុំដែន **នៃលេខរបស់អ្នកផ្ទាល់**។

ដោតវិធានរបស់អ្នកចូលក្នុងប្រព័ន្ធពិត

```
MY_SETTINGS = {"TEAM_PICKUP_MIN_PORTIONS": TEAM_PICKUP_MIN_PORTIONS,
               "URGENT_HOURS": URGENT_HOURS, "SOON_HOURS": SOON_HOURS}
my_system = fr.System(MY_SETTINGS,
                      hooks={"pickup_label": pickup_label, "urgency": urgency})
my_system.run_day(show=False)
print(my_system.report())
```

- **Hook** គឺជាកន្លែងដែលប្រព័ន្ធអនុញ្ញាតឱ្យអ្នកដោត function របស់អ្នកផ្ទាល់ចូល។
- បើ function របស់អ្នក crash ប្រព័ន្ធ print ⚠️ ហើយប្រើវិធាន default របស់វា។

🧭 ការសម្រេចចិត្តរបស់អ្នក · ទំហំក្រុម និងភាពបន្ទាន់

- ប្តូរ `TEAM_PICKUP_MIN_PORTIONS`, `URGENT_HOURS`, `SOON_HOURS` (ឬរក្សាវាទុក ហើយពន្យល់ហេតុផល)។
- Run ម្តងទៀត: ការពិនិត្យនៅតែជាប់ ហើយស្លាកប្តូរ។

កំណត់ត្រាការសម្រេចចិត្ត	
ជម្រើសរបស់ខ្ញុំ	team ចាប់ពី … ចំណែក; urgent ≤ … ម៉ោង; soon ≤ … ម៉ោង
ជម្រើសផ្សេង	…
ហេតុផលរបស់ខ្ញុំ	…
នរណាម្នាក់ផ្សេងវិញ	…

កិច្ចការសប្តាហ៍នេះ

- ⭐ **មូលដ្ឋាន:** P1–P4, បញ្ចាំងក្នុង Version A, `pickup_label` និង `urgency` ដែល ✅ ទាំងអស់ ហើយដោតចូលប្រព័ន្ធ, កំណត់ត្រាការសម្រេចចិត្ត។
- ⭐⭐ **បន្ថែម:** S1 `send_team` ដែលមានលក្ខខណ្ឌពីរ; `explain_label` ដែលប្រាប់មូលហេតុ។
- 🔎 **កិច្ចការស្រាវជ្រាវទី 2 (5%):** វិធានពីជីវិតពិត ដែលសរសេរឱ្យច្បាស់លាស់ ជាមួយករណីធម្មតា ករណីព្រំដែន និងករណីចម្លែក។ ចែករំលែកវានៅពេលចាប់ផ្តើមថ្នាក់ សប្តាហ៍ទី 3។

សប្តាហ៍ក្រោយ · Records និងទិន្នន័យ

សំណើមួយគឺងាយស្រួល។ មួយថ្ងៃពេញមានសំណើច្រើន។ អ្នកនឹងរក្សា **record មួយសម្រាប់សំណើនីមួយៗ** ឬកសរុបវា ហើយរកសំណើណាមួយតាម **ID** របស់វា។